MPOO 2027-1 · GRUPO 6 · PRÁCTICA 8 · MIÉRCOLES 21 DE OCTUBRE

La estación de clasificación

Herencia, segunda parte. Una banda transportadora con tres sensores y tres actuadores: primero la dibujas en un diagrama de clases y luego la programas en cuatro paquetes.

Descargar los archivos (.zip)

Qué vas a hacer

En pareja. Cuatro pasos:

1. **Dibuja el diagrama de clases**
   15 min · en papel o draw.io

2. **Escribe los cimientos: 4 clases**
   25 min · terminas cuando `PruebaTemperatura` imprime lo de la guía

3. **Escribe la estación: 7 clases más**
   45 min · terminas cuando `Main` imprime lo de la guía

4. **Entrega**
   viernes 23 de octubre, 23:59, en `p08/`

La historia

En una paquetería, los paquetes pasan por una **banda transportadora**. Tres **sensores** los vigilan: uno ve si hay un paquete atorado, otro lo pesa y otro mide la temperatura del motor. Tres **actuadores** responden: la banda, un **semáforo** y una **alarma**.

Si **cualquier** sensor da alerta, la banda se detiene, el semáforo se pone en rojo y suena la alarma. Si no, la banda avanza y el semáforo queda en verde.

Cada dispositivo trae **su propia batería**, que se gasta con el uso. La **estación** tiene hasta cinco sensores y cinco actuadores, y un sensor se puede sacar a mantenimiento y volver a instalar.

El mapa

Así queda todo cuando terminas. Tenlo a la vista en los cuatro pasos.

Quién hereda de quién, y en qué paquete vive

```
Dispositivo (abstract)        dispositivos  ◆ 1 Bateria
├── Sensor (abstract)         dispositivos
│   ├── SensorPresencia       sensores
│   ├── SensorPeso            sensores
│   └── SensorTemperatura     sensores
└── Actuador (abstract)       dispositivos
    ├── BandaTransportadora   actuadores
    ├── Semaforo              actuadores    ◆ 3 Luz
    └── Alarma                actuadores

Estacion   estacion      ◇ 1..5 Sensor y ◇ 1..5 Actuador
Bateria    dispositivos  ◆ la crea Dispositivo con new
Luz        dispositivos  ◆ las crea Semaforo con new
```

`├──` es **«es un»** (`extends`). ◆ **composición**: el dueño hace el `new` y la parte muere con él. ◇ **agregación**: la parte llega de afuera y sobrevive aunque la saquen.

Las carpetas en IntelliJ

```
src
├── dispositivos
│   ├── Bateria.java          paso 2
│   ├── Dispositivo.java      paso 2
│   ├── Sensor.java           paso 2
│   ├── Actuador.java         paso 3
│   └── Luz.java              viene en el .zip
├── sensores
│   ├── SensorTemperatura.javapaso 2
│   ├── SensorPresencia.java  paso 3
│   └── SensorPeso.java       paso 3
├── actuadores
│   ├── BandaTransportadora.javapaso 3
│   ├── Semaforo.java         paso 3
│   └── Alarma.java           paso 3
└── estacion
    ├── Estacion.java         paso 3
    ├── PruebaTemperatura.javaviene en el .zip
    └── Main.java             viene en el .zip, va al final
```

Para crear cada carpeta: clic derecho en `src` → **New → Package**. Cada archivo empieza con el `package` de su carpeta.

Paso 1 · El diagrama de clases

- **Una caja por clase del mapa** (son 12, sin `Main` ni `PruebaTemperatura`), con los atributos y métodos que dice cada clase en los pasos 2 y 3.
- **Los símbolos:** `+` public, `–` private, `#` protected, `~` sin modificador. Las clases y métodos `abstract`, en cursiva.
- **Las flechas, como en el mapa:** triángulo vacío hacia el padre; ◆ y ◇ del lado del dueño, con su número (`1`, `3`, `1..5`).

A los 15 minutos se proyecta la solución y corrigen el suyo.

Paso 2 · Los cimientos

Crea los cuatro paquetes y copia del .zip `Luz.java` (a `dispositivos`) y `PruebaTemperatura.java` (a `estacion`). **`Main.java` todavía no**: usa clases que aún no existen y nada correría.

Cada clase dice sus **atributos**, sus **constructores** y sus **métodos**; el código lo escribes tú. Una regla para todas: **atributos** **`private`**; clases, constructores y métodos **`public`**, salvo donde la clase diga otra cosa. Los nombres y los textos entre comillas, tal cual: `Main` los usa y la salida se compara letra por letra.

1 · **`Bateria`** en dispositivos

La batería de cada dispositivo. **Sin modificador** en la clase, el constructor y los métodos: solo la ve su paquete.

ATRIBUTOS

- `carga` (int): de 0 a 100.

CONSTRUCTOR · LLEVA UNO

- Recibe la `carga`. Si es menor que 0 guarda 0; si es mayor que 100, guarda 100.

MÉTODOS

- `getCarga()`: devuelve la carga.
- `descargar(puntos)`: resta esos puntos, sin bajar de 0.

2 · `Dispositivo` en dispositivos

El padre de todos. Es **abstracta**.

ATRIBUTOS

- `id` (String): no cambia nunca (`final`).
- `bateria` (Bateria): no cambia nunca (`final`). Es la ◆ composición.

CONSTRUCTORES · LLEVA DOS

- Con `id` y `carga`: guarda el id y crea su batería con `new Bateria(carga)`.
- Solo con `id`: no repite nada, llama al primero con `this(id, 100)`.

MÉTODOS

- `getId()`: devuelve el id.
- `getCarga()`: devuelve la carga de su batería.
- `gastar(puntos)`: descarga su batería. **`protected`**: solo para las hijas.
- `toString()`: devuelve `S3 (100%)`.

3 · `Sensor` en dispositivos

Un dispositivo que mide. **Abstracta**, hija de `Dispositivo`.

ATRIBUTOS

- `unidad` (String): `C`, `kg` o `cm`; no cambia nunca (`final`).
- `senal` (double): la última señal que recibió.

CONSTRUCTORES · LLEVA DOS

- Con `id`, `unidad` y `carga`: `super(id, carga)` y guarda la unidad.
- Con `id` y `unidad`: `this(id, unidad, 100)`.

MÉTODOS

- `getUnidad()`: devuelve la unidad.
- `recibir(senal)`: guarda la señal. Con él, `Main` simula el mundo real.
- `getSenal()`: devuelve la señal. `protected`: solo para las hijas.
- `medir()`: **abstracto**, devuelve un double.
- `hayAlerta()`: **abstracto**, devuelve un boolean.
- `leer()`: `final`, ninguna hija lo cambia. Gasta 1 de batería y devuelve `S3 mide 40.0 C`, y le agrega `ALERTA` si `hayAlerta()`.
- `toString()`: `Sensor` más lo del padre (`super.toString()`).

4 · `SensorTemperatura` en sensores

Hija de `Sensor`. Está en otro paquete: lleva `import dispositivos.Sensor;`.

ATRIBUTOS

- `calibracion` (double).
- `maxima` (double).

CONSTRUCTOR · LLEVA UNO

- Con `id`, `calibracion` y `maxima`: `super(id, "C")` y guarda los dos.

MÉTODOS

- `medir()`: la señal más la calibración.
- `hayAlerta()`: si `medir()` es mayor que la máxima.
- `toString()`: lo del padre más `temperatura, max 45.0`.

Corre `PruebaTemperatura`:

```
package estacion;

import sensores.SensorTemperatura;

public class PruebaTemperatura {
    public static void main(String[] args) {
        SensorTemperatura termo = new SensorTemperatura("S3", -1.5,
45);
        System.out.println(termo);
        termo.recibir(41.5);
        System.out.println(termo.leer());
        termo.recibir(50);
        System.out.println(termo.leer());
        System.out.println(termo);
    }
}
```

> **DEBE SALIR**
>
> ```
> Sensor S3 (100%) temperatura, max 45.0
> S3 mide 40.0 C
> S3 mide 48.5 C ALERTA
> Sensor S3 (98%) temperatura, max 45.0
> ```

El `98%` de la última línea es la batería: cada `leer()` gastó 1.

Paso 3 · La estación

Las demás clases, en este orden.

5 · `SensorPresencia` en sensores

Hija de `Sensor`. Ve qué tan cerca está el paquete.

ATRIBUTOS

- Ninguno nuevo.

CONSTRUCTOR · LLEVA UNO

- Con `id`: `super(id, "cm")`.

MÉTODOS

- `medir()`: la señal llega en milímetros; devuelve centímetros (entre 10).
- `hayAlerta()`: si `medir()` es menor que 2: el paquete se atoró.
- `toString()`: lo del padre más `presencia`.

6 · `SensorPeso` en sensores

Hija de `Sensor`. La báscula.

ATRIBUTOS

- `limite` (double): en kilos. Si le dan 0 o menos, guarda 1.

CONSTRUCTORES · LLEVA DOS

- Con `id`, `carga` y `limite`: `super(id, "kg", carga)` y guarda el límite.
- Con `id` y `limite`: `this(id, 100, limite)`.

MÉTODOS

- `medir()`: la señal llega en gramos; devuelve kilos (entre 1000).
- `hayAlerta()`: si `medir()` es mayor que el límite.
- `toString()`: lo del padre más `peso, limite 25.0`.

7 · `Actuador` en dispositivos

Un dispositivo que hace algo. **Abstracta**, hija de `Dispositivo` y hermana de `Sensor`.

ATRIBUTOS

- `encendido` (boolean): nace en false.

CONSTRUCTORES · LLEVA DOS

- Con `id` y `carga`: `super(id, carga)`.
- Solo con `id`: `this(id, 100)`.

MÉTODOS

- `isEncendido()`: devuelve si está encendido.
- `encender()` y `apagar()`: lo cambian.
- `reaccionar(alerta)`: **abstracto**. Recibe un boolean y devuelve un String con lo que hizo.
- `toString()`: `Actuador` más lo del padre.

8 · `Alarma` en actuadores

Hija de `Actuador`. Lleva `import dispositivos.Actuador;`.

ATRIBUTOS

- `volumen` (int): de 0 a 10.

CONSTRUCTOR · LLEVA UNO

- Con `id` y `volumen`: `super(id)`. Volumen menor que 0 guarda 0; mayor que 10, guarda 10.

MÉTODOS

- `getVolumen()`: devuelve el volumen.
- `reaccionar(alerta)`: con alerta, `encender()` y devuelve `A3 suena a volumen 10`. Sin alerta, `apagar()` y devuelve `A3 en silencio`.
- `toString()`: lo del padre más `alarma, volumen 10`.

9 · `BandaTransportadora` en actuadores

Hija de `Actuador`. Mueve los paquetes.

ATRIBUTOS

- `velocidad` (int): metros por ciclo. Si le dan menos de 1, guarda 1.
- `recorrido` (int): metros que lleva en total.

CONSTRUCTOR · LLEVA UNO

- Con `id` y `velocidad`: `super(id)` y guarda la velocidad.

MÉTODOS

- `getRecorrido()`: devuelve el recorrido.
- `avanzar(metros)`: si está apagada, devuelve `A1 apagada, no avanza`. Si está encendida, suma los metros al recorrido, `gastar(metros)` y devuelve `A1 avanza 2 m, lleva 2 m`.
- `avanzar()`: sin parámetros. Llama al otro con su velocidad. Es **sobrecarga**.
- `reaccionar(alerta)`: con alerta, `apagar()` y devuelve `A1 se detiene`. Sin alerta, `encender()` y devuelve lo de `avanzar()`.
- `toString()`: lo del padre más `banda, 2 m por ciclo`.

10 · `Semaforo` en actuadores

Hija de `Actuador`. Importa `Actuador` y `Luz` (que ya viene en el .zip).

ATRIBUTOS

- `luces` (arreglo de `Luz`): no cambia (`final`). Es la ◆ composición.

CONSTRUCTOR · LLEVA UNO

- Con `id`: `super(id)`, crea el arreglo de 3 y adentro tres `new Luz`: `rojo`, `amarillo` y `verde`.

MÉTODOS

- `ponerEn(color)`: enciende la luz de ese color y apaga las otras (compara con `equals`).
- `getColor()`: devuelve el color de la luz encendida, o `apagado`.
- `reaccionar(alerta)`: `encender()`, `ponerEn("rojo")` con alerta o `ponerEn("verde")` sin ella, y devuelve `A2 en verde`.
- `toString()`: lo del padre más `semaforo en verde`.

11 · `Estacion` en estacion

No hereda de nadie. Importa `Sensor` y `Actuador`.

ATRIBUTOS

- `nombre` (String): no cambia (`final`).
- `sensores` (arreglo de `Sensor`) y `actuadores` (arreglo de `Actuador`): la ◇ agregación.
- `numSensores` y `numActuadores` (int): cuántos hay instalados.
- `ciclos` (int).

CONSTRUCTOR · LLEVA UNO

- Con `nombre`: guarda el nombre y crea los dos arreglos de tamaño 5, **vacíos**. Los dispositivos no se crean aquí: llegan de `Main`.

MÉTODOS

- `instalar(Sensor s)` e `instalar(Actuador a)`: dos métodos con el mismo nombre (**sobrecarga**). Lo ponen en el siguiente lugar libre e imprimen `Instalado:` más el dispositivo. Si ya no cabe: `Sin lugar para S1`.
- `retirar(id)`: devuelve un `Sensor`. Lo busca por id, recorre los que siguen un lugar a la izquierda, imprime `Retirado: S2` y lo devuelve. Si no está, devuelve `null`.
- `ciclo()`: suma un ciclo e imprime `== Ciclo 1 en Anden 3`. Imprime `  ` más el `leer()` de cada sensor; si **alguno** tiene `hayAlerta()`, hay alerta. Después imprime `  ` más el `reaccionar(alerta)` de cada actuador.
- `toString()`: devuelve `Anden 3: 3 sensores, 3 actuadores, 3 ciclos`.

Con todo listo, copia `Main.java` a `estacion` y córrelo:

```
package estacion;

import actuadores.Alarma;
import actuadores.BandaTransportadora;
import actuadores.Semaforo;
import dispositivos.Sensor;
import sensores.SensorPeso;
import sensores.SensorPresencia;
import sensores.SensorTemperatura;

public class Main {
    public static void main(String[] args) {
        // los dispositivos nacen aquí, fuera de la estación
        SensorPresencia ojo = new SensorPresencia("S1");
        SensorPeso bascula = new SensorPeso("S2", 80, 25);
        SensorTemperatura termo = new SensorTemperatura("S3", -1.5,
45);
        BandaTransportadora banda = new BandaTransportadora("A1", 2);
        Semaforo semaforo = new Semaforo("A2");
        Alarma alarma = new Alarma("A3", 15);

        Estacion anden = new Estacion("Anden 3");
        anden.instalar(ojo);
        anden.instalar(bascula);
        anden.instalar(termo);
        anden.instalar(banda);
        anden.instalar(semaforo);
        anden.instalar(alarma);

        // un paquete normal
        ojo.recibir(120);
        bascula.recibir(18500);
        termo.recibir(41.5);
        anden.ciclo();

        // un paquete con sobrepeso
```

```
        ojo.recibir(95);
        bascula.recibir(32000);
        termo.recibir(43.5);
        anden.ciclo();

        // la báscula sale a mantenimiento y el motor se calienta
        Sensor fuera = anden.retirar("S2");
        ojo.recibir(150);
        termo.recibir(48);
        anden.ciclo();

        // la báscula regresa: es el mismo objeto
        System.out.println("Regresa: " + fuera);
        anden.instalar(fuera);
        banda.encender();
        System.out.println(banda.avanzar(5));
        System.out.println(anden);
    }
}
```

```
DEBE SALIR
Instalado: Sensor S1 (100%) presencia
Instalado: Sensor S2 (80%) peso, limite 25.0
Instalado: Sensor S3 (100%) temperatura, max 45.0
Instalado: Actuador A1 (100%) banda, 2 m por ciclo
Instalado: Actuador A2 (100%) semaforo en apagado
Instalado: Actuador A3 (100%) alarma, volumen 10
== Ciclo 1 en Anden 3
  S1 mide 12.0 cm
  S2 mide 18.5 kg
  S3 mide 40.0 C
  A1 avanza 2 m, lleva 2 m
  A2 en verde
  A3 en silencio
== Ciclo 2 en Anden 3
  S1 mide 9.5 cm
  S2 mide 32.0 kg ALERTA
  S3 mide 42.0 C
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Retirado: S2
== Ciclo 3 en Anden 3
  S1 mide 15.0 cm
  S3 mide 46.5 C ALERTA
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Regresa: Sensor S2 (78%) peso, limite 25.0
Instalado: Sensor S2 (78%) peso, limite 25.0
A1 avanza 5 m, lleva 7 m
Anden 3: 3 sensores, 3 actuadores, 3 ciclos
```

En el ciclo 2, **un solo** sensor en alerta detiene todo. En el ciclo 3 la estación funciona sin la báscula, y la báscula regresa con `78%`: es **el mismo objeto**, con la batería que ya había gastado.

Si algo truena

No compila

- `SensorPresencia is not abstract and does not override abstract method hayAlerta() in Sensor`: a tu sensor le falta uno de los métodos abstractos del padre. Una hija concreta tiene que escribir **todos**: `medir()` y `hayAlerta()`.
- `Sensor is abstract; cannot be instantiated`: `Sensor` es abstracta: no hay sensores a secas, solo de presencia, de peso o de temperatura.
- `call to this must be first statement in constructor`: el `this(...)`, igual que el `super(...)`, tiene que ser la primera línea del constructor.
- `senal has private access in Sensor`: la señal es privada de `Sensor`. Las hijas la leen con `getSenal()`, que es `protected`.
- `Semaforo is not public in actuadores; cannot be accessed from outside package`: a la clase le falta `public`. Sin modificador solo la ve su paquete, y `Main` vive en `estacion`.
- `Bateria is not public in dispositivos; cannot be accessed from outside package`: quisiste crear una batería desde otro paquete. La batería la crea `Dispositivo` y nadie más.
- `leer() in SensorPeso cannot override leer() in Sensor`: `leer()` es `final` en `Sensor`: las hijas no la sobrescriben, solo escriben `medir()` y `hayAlerta()`.
- `cannot find symbol`: falta un `import`. Cada clase de otro paquete se importa por su nombre completo, por ejemplo `import actuadores.Alarma;`.

Compila, pero la salida no es la de la guía

- **En el ciclo 2 la banda sigue avanzando**: tu `ciclo()` se queda con la alerta del último sensor. Lee **todos** los sensores primero y, si **alguno** dio alerta, hay alerta.
- **En el ciclo 3 vuelve a salir `S2` y falta `S3`**: tu `retirar` resta el contador pero no recorre los demás a la izquierda.
- **Sale `Regresa: null`**: tu `retirar` borra el sensor antes de guardarlo para devolverlo.
- **La alarma dice `volumen 15`**: el constructor de `Alarma` no valida.

Paso 4 · La entrega

QUÉ	DETALLE
Cuándo	**Viernes 23 de octubre, 23:59.** Sin IA, como siempre.
Con quién	En pareja (o de tres si alguien se queda solo). **Cada integrante sube todo en su propia carpeta.**
Dónde	Tu rama `entregas_apellido_nombre`, carpeta `entregas/apellido_nombre/p08/`. Solo push.
Qué	Tu carpeta `src`, la carpeta `out` (los `.class`) y una **bitácora en PDF**

La bitácora lleva: los integrantes (nombre y número de cuenta), la foto o captura de **tu diagrama**, la captura de la **salida de `Main`**, y estas tres respuestas con tus palabras:

1. **Sobrecarga contra sobrescritura:** un ejemplo de cada una sacado de tu código, y la diferencia.
2. **Composición contra agregación:** ¿por qué la `Bateria` es composición y el sensor en la `Estacion` es agregación? Piensa en quién hace el `new` y en qué le pasó a `S2` en el ciclo 3.
3. **Modificadores:** agrega en `Main` la línea `bascula.gastar(5);`, copia el error y explica por qué `SensorPeso` sí puede usar `gastar` y `Main` no. Bórrala antes de entregar.

Cómo se califica, sobre 10:

QUÉ REVISO	PUNTOS
Diagrama: las 12 cajas con sus símbolos, herencia, ◆, ◇ y multiplicidad	2
Corre y la salida es la de la guía	2
Constructores: `super(...)` en cada hija y `this(...)` en los constructores que no repiten código	2
Sobrecarga y sobrescritura: `instalar`, `avanzar` y los `@Override`	1.5
Modificadores y paquetes: atributos `private`, lo que solo usan las hijas `protected`, `Bateria` sin modificador, cada clase en su paquete del mapa	1.5
Bitácora: las tres respuestas	1

```
git checkout entregas_apellido_nombre
git pull origin main
mkdir -p entregas/apellido_nombre/p08
# copia aquí src, out y la bitácora en PDF
git add entregas/apellido_nombre/p08
git commit -m "P8: herencia, segunda parte"
git push
```

Si los `.class` no aparecen en `git status`, usa `git add -f`. El código tiene que ser suyo; si algo no sale, entréguenlo igual y cuenten en la bitácora dónde se atoraron.
